Autonomous And Adaptive Systems - Project Report

Alessandro Tutone
Alma Mater Studiorum - University of Bologna
alessandro.tutone@studio.unibo.it

Abstract

This project is about the implementation of a reinforcement learning algorithm to learn a cooperative behavior in a multi-agent system. The environment used is Overcooked-AI, where two agents must coordinate actions to maximize common cumulative rewards. Starting from a simple one-step Actor-Critic method that struggles in achieving stable performance, the algorithm implemented at the end was an n-step Actor-Critic that was able to learn a proper policy that consistently performed well in each testing episode. Several key implementation challenges, including reward shaping, observation design, and catastrophic forgetting, are analyzed and discussed. Learning a good policy was not immediate, indeed, it requires many hours of training, but in the end satisfactory results were achieved.

1 Introduction

Overcooked-AI Carroll et al. [2019] is a benchmark environment for multi-agent cooperative task performance, based on the wildly popular video game Overcooked. The goal of the game is to serve as many soups as possible within the time limit. The process of delivering a soup requires the chefs to grab three onions, put them inside the pot, wait for the soup to be ready, grab a plate, fill the plate with the cooked soup, and deliver the final dish to the correct spot. The agents should learn how to coordinate effectively within the environment, avoiding collisions, and splitting the required tasks in the best possible way to maximize the final cumulative reward.

The reward is shared between the two agents, that is, if one agent manages to correctly deliver a soup, the reward is also received by the other agent. However, the real challenge related to rewards is linked to the fact that delivering a soup is the result of a sequence of actions that are quite difficult to find through random exploration. This problem was addressed with reward shaping that allows agents to learn intermediate steps before learning the complete sequence of preparing and delivering a soup.

Learning this kind of behavior might be complicated with simple algorithms; indeed, a simple DQN might struggle in converging towards an optimal policy. A group of students from the University of Southern California published a paper Cai et al. [2020] in which they analyzed three methods to complete the task proposed by the Overcooked-AI environment: *Advantage Actor Critic* Mnih et al. [2016] (A2C), a widely used reinforcement learning algorithm, *Proximal Policy Optimization* Schulman et al. [2017] (PPO), a more recent method, and Behavior Cloning Ly and Akhloufi [2020], which consists in learning policy from human data. Looking at the promising results achieved by this research team, as a first step to complete this project work, a simple implementation of the Actor-Critic algorithm was written. The first method implemented as a baseline was the one-step Actor–Critic described in the book "Reinforcement Learning: An Introduction" Sutton et al. [1998]. Since this naive approach did not achieve satisfactory performance, several modifications were introduced to significantly improve the results.

38th Conference on Neural Information Processing Systems (NeurIPS 2024).

2 Method

The first step to achieve good performance was implementing a basic algorithm that could learn some kind of policy. Starting from the performance achieved, the next step was to improve that method in order to obtain better results each time. Considering the task it was needed to complete, a good starting point was the one-step Actor-Critic method.

2.1 One-step Actor-Critic

The main algorithm was written following the pseudo-code in the book by Sutton and Barto Sutton et al. [1998] (see Figure 1).

One-step Actor–Critic (episodic), for estimating $\pi_\theta \approx \pi_*$

Input: a differentiable policy parameterization $\pi(a|s,\boldsymbol{\theta})$
Input: a differentiable state-value function parameterization $\hat{v}(s,\mathbf{w})$
Parameters: step sizes $\alpha^{\boldsymbol{\theta}} > 0$, $\alpha^{\mathbf{w}} > 0$
Initialize policy parameter $\boldsymbol{\theta} \in \mathbb{R}^{d'}$ and state-value weights $\mathbf{w} \in \mathbb{R}^{d}$ (e.g., to $\mathbf{0}$)
Loop forever (for each episode):
  Initialize S (first state of episode)
  $I \leftarrow 1$
  Loop while S is not terminal (for each time step):
    $A \sim \pi(\cdot|S,\boldsymbol{\theta})$
    Take action A, observe S', R
    $\delta \leftarrow R + \gamma \hat{v}(S',\mathbf{w}) - \hat{v}(S,\mathbf{w})$ (if S' is terminal, then $\hat{v}(S',\mathbf{w}) \doteq 0$)
    $\mathbf{w} \leftarrow \mathbf{w} + \alpha^{\mathbf{w}} \delta \nabla \hat{v}(S,\mathbf{w})$
    $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \alpha^{\boldsymbol{\theta}} I \delta \nabla \ln \pi(A|S,\boldsymbol{\theta})$
    $I \leftarrow \gamma I$
    $S \leftarrow S'$

Figure 1: One-step Actor-Critic algorithm.

For the implementation, two networks were used:

- *Actor* - A 3-layer neural network that takes as input the state of the environment and returns a probability distribution over the action space.
- *Critic* - A neural network with the same structure as the *actor* one that takes as input the state of the environment and returns an estimate of the reward.

Once the implementation was completed, the model was trained using the *featurized state encoding*, which is a vector that captures general information about the actual state of the environment (number of onions in the pot, state of the pot, agent orientation, etc.). Looking at the official documentation, the meaning of each value was explained; therefore, it could be a reasonable idea to use it as observation.

After training for several episodes, little or no improvement was made throughout the execution. The main problems that could lead to such a bad result are the following:

1. The method used was too simple to learn a proper policy: since the actions that contribute to the final results are not close in time, it might be a good idea to implement the *n-step* version of the algorithm to take into account a longer sequence of steps.
2. Problems related to the reward: the fact that the reward was received only after a long sequence of steps could be a problem for the *actor* to learn the policy. Doing reward shaping to introduce intermediate rewards could be useful to lead the agents towards their final goal (delivering a soup).
3. Wrong network structure: the use of two distinct networks for *actor* and *critic* might lead to a slow convergence. A different common implementation choice was the one used by TensorFlow TensorFlow [2024]; using a combined *actor-critic* network, instead of two different ones.

2.2 n-step Actor-Critic

Given the previous implementation, going from the *one-step* to the *n-step* update was quite straightforward, even if some adjustments were required. Compared to the previous algorithm, in this case, a buffer was needed to save the *n* steps to perform the update. With the *one-step update*, the correction of the weights of the networks was performed after each step, while with the *n-step update* a longer sequence of steps is taken into account.

$$G_{t:t+n} \doteq R_{t+1} + \gamma R_{t+2} + ... + \gamma^{n-1} R_{t+n} + \gamma^n \hat{v}(S_{t+n}, \mathbf{w})$$

$$\delta \doteq G_{t:t+n} - \hat{v}(S_t, \mathbf{w})$$

Considering the fact that the final result was the consequence of many steps and not just one single action, looking at a longer chain of rewards might lead the agents to learn the correct policy faster. The one-step update rule, which relies solely on the immediate reward and the value of the very next state, is unable to effectively credit individual actions that contribute to a distant future reward. Using an n-step return, the algorithm can take into account actions that are distant in time from the reward received.

Implementing the *n-step* version was achieved by following the *Chapter 7: n-step Bootstrapping* of the book "Reinforcement Learning: An Introduction" Sutton et al. [1998]. After improving the algorithm, the results obtained were not yet satisfactory. Even if many training episodes were performed, the two agents managed to deliver only one or two soups per episode. The training process was slow, and increasing the learning rate only led to worse performance.

A probable cause of this slow learning process was the architecture of the two neural networks since they were designed on the spot before implementing the algorithms, and then they were kept as they were. The two networks were really basic, just 3 layers with a relatively low hidden size. Another relevant problem was related to the observations that the agents were using: the *featurized state encoding* includes a lot of information, but it was not enough for the agents to learn how to move around the environment. Changing to the *lossless state encoding* was a good idea.

2.3 Networks and observation design

After executing the commands "`env.reset()`" and "`env.step()`", the observation variable returned contains two different encodings Cai et al. [2020]: the *featurized state encoding* (used until now) and the *lossless state encoding*, which consists of 26 binary matrices, each including a certain aspect of the state (chefs' position, dishes' position, delivering spot, ecc.).

Assuming that the encoding used until now was not sufficient for the chefs to learn, the *lossless state encoding* was used as an alternative. Having now a two-dimensional input, the best thing to do was to use a CNN to avoid losing the spatial information. The architecture of the two networks, the *actor* and the *critic*, was taken from the original paper Carroll et al. [2019]: 3 convolutional layers (of sizes 5×5, 3×3, and 3×3 respectively), each of which has 25 filters, followed by 3 fully-connected layers with hidden size 32.

After many hours of training, the results reached were still not sufficient. The first soups were delivered after several attempts, and converging to a sound policy seemed impossible. The last thing it was tried was *reward shaping*.

2.4 Reward shaping

The preparation and delivery of a soup is the result of a sequence of actions. Guessing the right sequence by behave stochastically is quite hard. Adding intermediate rewards might be useful to the agent in learning the proper sequence of actions that lead to the final reward. To perform reward shaping, the *featurized state encoding* was used, since it contains all the information needed to modify the reward.

Finding appropriate intermediate rewards was not straightforward, but the final combination of rewards and penalties was the following:

- **+0.2** - For each step the soup was in the *"cooking"* state
- **-0.05** - For each step the soup was in the *"ready"* state

Rewarding the agents for filling the pot with onions leads them to understand how to get the onions and place them in the correct spot (inside the empty pot). On the other hand, once the soup was ready, penalizing the soup in the "*ready*" state was done to push the agent to remove the soup with a dish as soon as possible, without wasting time.

3 Experiments

Once the algorithm implemented started to show some kind of improvement with respect to the baseline previously implemented, a long training consisting of several training sessions was carried out. All training was performed on the Kaggle platform Kaggle [2025], and the weights were saved every 10 episodes of training. Taking into account the Kaggle limitations (each session ends after 10 hours), each training was composed of 300 episodes. The highest results were obtained after 3300 training episodes, and the cumulative reward received after that point in each episode can be seen in Figure 2.

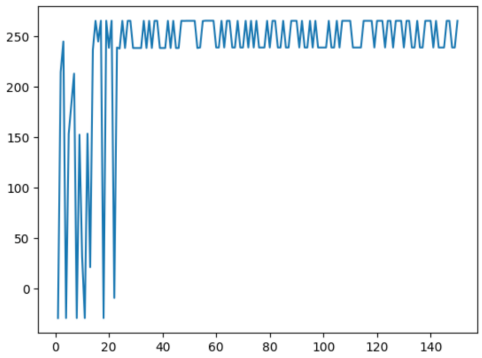

Figure 2: Best results reached.

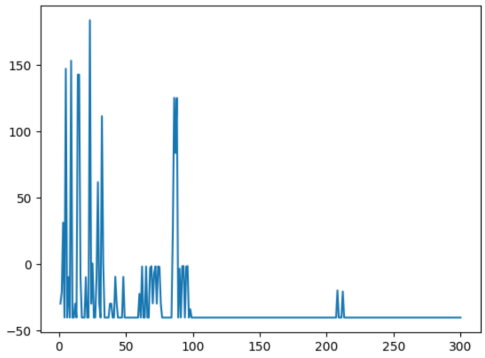

Figure 3: Problem of forgetting.

After reaching a policy that was close to the optimal one, if training was continued, the agents forgot all what they have learned. The two chefs have experienced the *catastrophic forgetting problem*, as can be seen in Figure 3. Therefore, the weights used for the testing part are the ones that has been saved before forgetting, around episode 3300.

4 Conclusion

Even if the *Overcooked-AI* environment was a simplification of more complicated multi-agent systems, it helped me see in practice those algorithms that before were just pure theory. The initial idea was to implement the most basic algorithm possible to see if more advanced methods such as PPO Schulman et al. [2017] and A2C Mnih et al. [2016] are always needed to solve certain kinds of task. Given some initial problems related to my inexperience, more and more adjustments were made to the initial baseline, deviating from the original idea to keep everything as simple as possible. Considering the effect received, the game-changing modification was the introduction of intermediate rewards, since after that the learning improved drastically.

Other types of implementations could have been interesting to analyze, such as the use of two distinct networks for the two agents, or the use of a CNN that takes as input observations of different dimensions. A limitation of this study is the inability to test generalization due to the fixed-dimension input of the CNN architecture, which was designed specifically for the *cramped_room* layout. The CNN ability to handle inputs of different sizes was not properly exploited during the implementation of the networks. A further improvement might be the correction of this issue in order to train and test the algorithm with a layout different from the first one.

References

Zikai Cai, Muchen Ge, Zilong Guo, Yufen Huang, An Liu, Huating Wen, and Shengxiang Xu. Deep learning based overcooked play agent. 2020.

Micah Carroll, Rohin Shah, Mark K Ho, Tom Griffiths, Sanjit Seshia, Pieter Abbeel, and Anca Dragan. On the utility of learning about humans for human-ai coordination. *Advances in neural information processing systems*, 32, 2019.

Kaggle. Kaggle: Your machine learning and data science community. `https://www.kaggle.com`, 2025. Accessed: 2025-08-24.

Abdoulaye O Ly and Moulay Akhloufi. Learning to drive by imitation: An overview of deep behavior cloning methods. *IEEE Transactions on Intelligent Vehicles*, 6(2):195–209, 2020.

Volodymyr Mnih, Adria Puigdomenech Badia, Mehdi Mirza, Alex Graves, Timothy Lillicrap, Tim Harley, David Silver, and Koray Kavukcuoglu. Asynchronous methods for deep reinforcement learning. In *International conference on machine learning*, pages 1928–1937. PmLR, 2016.

John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

Richard S Sutton, Andrew G Barto, et al. *Introduction to reinforcement learning*, volume 135. MIT press Cambridge, 1998.

TensorFlow. Playing cartpole with the actor-critic method. `https://www.tensorflow.org/tutorials/reinforcement_learning/actor_critic`, 2024. Accessed: 2025-08-24.